Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

ER-AFD

Convertidor de ER a AFD

Proyecto del curso — Documentación Técnica y Manual de Usuario

Integrantes del grupo:	**Carné:**
Keila Daniela Silva Aguilar	5190-24-1964
Yefri Osvaldo Castillo Pérez	0905-22-1273
Henderson Alexander Chalí España	1990-24-5916
Diego Martín Flores González	3190-23-8699

Catedrático: Ing. René Ornélyz

Guatemala, noviembre de 2026

Contenido

1. Introducción y descripción general del proyecto 3
2. Objetivos y restricciones cumplidas 3
3. Módulo de entrada de expresiones regulares 4
3.1 Operadores, conjuntos y secuencias de escape soportadas 4
3.2 Tokenización y validación sintáctica (Parser descendente recursivo) 5
4. Módulo de conversión a AFND y AFD 5
4.1 Algoritmo de Thompson (ER a AFND) 5
4.2 Construcción de subconjuntos y cerradura épsilon (AFND a AFD) 6
4.3 Desarrollo completo del autómata para la expresión (a|b)*abb 7
5. Módulo de análisis de entrada y casos de prueba 8
5.1 Prueba de aceptación y retorno de código 1 8
5.2 Prueba de rechazo y retorno de código 0 9
5.3 Pruebas con comodines, conjuntos negativos y secuencias de escape 10
6. Manual de usuario e instalación 11
7. Código fuente y archivo Makefile 12

1. Introducción y descripción general del proyecto

El presente documento describe el diseño, implementación y pruebas del proyecto ER-AFD, desarrollado en el lenguaje Python para el curso de Autómatas y Lenguajes Formales. La herramienta tiene como finalidad procesar una expresión regular (ER) ingresada desde la línea de comandos, transformarla inicialmente en un Autómata Finito No Determinista (AFND) mediante el algoritmo de Thompson y posteriormente convertir dicho autómata en un Autómata Finito Determinista (AFD) empleando el método de construcción de subconjuntos.

Una vez generadas y desplegadas en consola las tablas de transiciones tanto del AFND como del AFD, el programa evalúa la cadena de entrada carácter por carácter sobre la matriz de transiciones del AFD. Durante la evaluación se imprime en pantalla cada una de las transiciones de estado efectuadas y se emite el resultado final indicando si la cadena fue aceptada o rechazada, devolviendo al sistema operativo el código de retorno 1 en caso de aceptación y 0 en caso de rechazo.

2. Objetivos y restricciones cumplidas

2.1 Objetivo principal

Aplicar los conceptos teóricos de conversión de expresiones regulares a autómatas finitos deterministas (AFD) para la solución de problemas de reconocimiento de patrones en cadenas de texto.

2.2 Objetivos específicos

- Implementar la conversión secuencial de una expresión regular hacia un autómata finito por medio de la ruta ER -> AFND -> AFD.
- Generar y desplegar en consola la tabla de transiciones, el alfabeto y los estados finales tanto del AFND como del AFD siguiendo el formato especificado.
- Evaluar cadenas de texto de entrada para determinar si son aceptadas o rechazadas por el autómata generado, mostrando la traza del recorrido de estados.

2.3 Cumplimiento de restricciones

Para cumplir con las restricciones establecidas en la hoja del proyecto, el núcleo del convertidor y evaluador se programó íntegramente utilizando estructuras de datos estándar de Python (listas, diccionarios, conjuntos y colas). En ningún módulo del programa se hace uso de la librería nativa 're' ni de ninguna otra biblioteca externa de expresiones regulares. El análisis sintáctico se resolvió mediante un parser descendente recursivo propio y la verificación funcional se complementó con pruebas unitarias automatizadas en pytest.

3. Módulo de entrada de expresiones regulares

3.1 Operadores, conjuntos y secuencias de escape soportadas

El analizador léxico recorre la expresión regular ingresada por el usuario e identifica los operadores básicos de concatenación implícita, unión (|), cerradura de Kleene (*) y agrupación con paréntesis (), así como los conjuntos y secuencias de escape requeridos en el alcance funcional:

Símbolo / Secuencia	Descripción e implementación en el sistema
.	Cualquier símbolo. Se mantiene como símbolo '.' en el alfabeto Σ cuando aparece en la ER.
ε	Símbolo nulo (épsilon). Si aparece en la ER se respeta su posición en Σ; si no, el AFND lo agrega al final del alfabeto.
*****	Operador de cerradura de Kleene (cero o más repeticiones).
|	Operador de unión o alternancia entre dos subexpresiones.
ab	Concatenación implícita entre símbolos o subexpresiones adyacentes.
(r)	Paréntesis para agrupar y dar mayor precedencia a la subexpresión r.
[abc]	Operador de conjunto positivo; internamente equivale a la unión (a|b|c).
[^abc]	Conjunto negativo; acepta cualquier carácter que no sea a, b ni c.
[a-z]	Conjunto con rango; abarca todos los caracteres ASCII desde el inicio hasta el final del rango.
\	Secuencia de escape literal; interpreta el siguiente carácter como símbolo normal (por ejemplo *, \\, \p).
\N	Secuencia de escape para un dígito; equivale al rango [0-9].
\hXX	Secuencia de escape para introducir un carácter por su código ASCII en hexadecimal (ej. \h20 para espacio, \h41 para A).
\A	Secuencia de escape para cualquier letra mayúscula; equivale a [A-Z].
\a	Secuencia de escape para cualquier letra minúscula; equivale a [a-z].
\z	Secuencia de escape para cualquier letra mayúscula o minúscula; equivale a [a-zA-Z].

3.2 Tokenización y validación sintáctica (Parser descendente recursivo)

Durante el análisis léxico (función tokenize_regex), el programa registra los símbolos en el alfabeto Σ en el mismo orden en que van apareciendo dentro de la expresión regular. Debido a que la concatenación no tiene un símbolo visible en la entrada del usuario, el tokenizador inserta un operador interno CONCAT cada vez que detecta dos elementos contiguos que deben concatenarse (por ejemplo, un símbolo o cierre '*' seguido de otro símbolo o de un paréntesis de apertura).

Con la lista de tokens generada, la clase RecursiveDescentParser valida la sintaxis y construye el Árbol de Sintaxis Abstracta (AST) aplicando la jerarquía de precedencia estándar (paréntesis, luego cerradura de Kleene, luego concatenación y por último unión) mediante las siguientes reglas gramaticales:

Gramática implementada en el parser descendente recursivo:

```
expr   -> term ( '|' term )*
term   -> factor ( 'CONCAT' factor )*
factor -> base ( '*' )*
base   -> SYM | '(' expr ')'
```

4. Módulo de conversión a AFND y AFD

4.1 Algoritmo de Thompson (ER a AFND)

La conversión del árbol sintáctico hacia el Autómata Finito No Determinista se realiza en la función build_thompson_afnd. Los estados del AFND se numeran de forma correlativa como q0, q1, q2, ..., qn, donde q0 es el estado inicial y el último estado creado corresponde al único estado de aceptación F. Para cada nodo del árbol se aplican las construcciones formales de Thompson:

- Para un símbolo básico (SymbolNode), se crea un nuevo estado de salida y se enlaza desde el estado actual mediante una transición etiquetada con dicho símbolo.
- Para una concatenación (ConcatNode), el estado final del subautómata izquierdo se utiliza directamente como estado inicial del subautómata derecho, evitando estados redundantes.
- Para una unión (UnionNode), se crean dos ramas paralelas que parten desde el estado actual mediante transiciones épsilon (ε) y convergen en un nuevo estado de salida común también con transiciones épsilon.
- Para una cerradura de Kleene (StarNode), se agregan las transiciones épsilon que permiten entrar al subautómata interno, repetirlo regresando a su inicio, o saltar directamente hacia el estado de salida para aceptar cero repeticiones.

4.2 Construcción de subconjuntos y cerradura épsilon (AFND a AFD)

Una vez obtenido el AFND, la función build_subset_afd aplica el algoritmo de construcción de subconjuntos para obtener un AFD equivalente cuyos estados se identifican como Q0, Q1, ..., Qm (siendo Q0 el estado inicial). El procedimiento se apoya en dos operaciones fundamentales:

- **1.** Cerradura épsilon (eps_closure): dado un conjunto de estados del AFND, calcula todos los estados a los que se puede llegar recorriendo únicamente transiciones ε.
- **2.** Función mover (move): dado un subconjunto de estados T y un símbolo del alfabeto a (distinto de ε), obtiene los estados destino de todas las transiciones con el símbolo a que parten desde algún estado en T.

El algoritmo comienza calculando Q0 = eps_closure({q0}) y lo coloca en una cola de estados pendientes. Para cada subconjunto Qi extraído de la cola y para cada símbolo del alfabeto del AFD, se calcula el nuevo conjunto U = eps_closure(move(Qi, símbolo)). Si U no está vacío y aún no ha sido registrado, recibe el siguiente identificador correlativo Qj y se agrega a la cola. Finalmente, cualquier estado Qi que contenga en su interior al estado de aceptación del AFND es incluido dentro del conjunto de estados de aceptación F del AFD.

4.3 Desarrollo completo del autómata para la expresión (a|b)*abb

Como caso representativo del funcionamiento de los algoritmos implementados, se muestra la conversión detallada de la expresión regular (a|b)*abb. El algoritmo de Thompson genera 11 estados en el AFND (de q0 a q10, con estado de aceptación F={q10}). Al aplicar la construcción de subconjuntos sobre el alfabeto Σ={a,b}, se obtienen los siguientes 5 estados deterministas en el AFD:

Estado del AFD	Subconjunto de estados del AFND	Con 'a'	Con 'b'	Aceptación
Q0 (Inicial)	{q0, q1, q2, q4, q7}	Q1	Q2	No
Q1	{q1, q2, q3, q4, q6, q7, q8}	Q1	Q3	No
Q2	{q1, q2, q4, q5, q6, q7}	Q1	Q2	No
Q3	{q1, q2, q4, q5, q6, q7, q9}	Q1	Q4	No
Q4 (Final)	{q1, q2, q4, q5, q6, q7, q10}	Q1	Q2	Sí (F = {Q4})

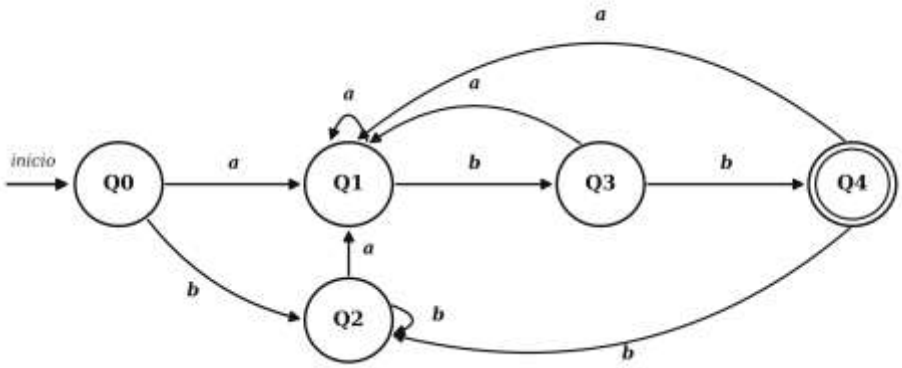


*Figura 1. Diagrama de transiciones del AFD generado para la expresión regular (a|b)*abb.*

5. Módulo de análisis de entrada y casos de prueba

El módulo de evaluación recorre la cadena de prueba desde el estado inicial Q0 consultando en cada paso la tabla de transiciones del AFD. A continuación se presentan las pruebas ejecutadas en consola junto con sus respectivas capturas de pantalla.

5.1 Prueba de aceptación y retorno de código 1

Se ejecutó el programa con la expresión regular (a|b)*abb y la cadena de entrada "aabb". El sistema construyó el AFND y el AFD, realizó el recorrido Q0 -> Q1 -> Q1 -> Q3 -> Q4 y, dado que Q4 pertenece al conjunto de estados finales F={Q4}, mostró el mensaje "Cadena aceptada" retornando el código de salida 1.

Salida completa en consola para la entrada "(a|b)*abb" "aabb":

```
AFND
Q={q0,q1,q2,q3,q4,q5,q6,q7,q8,q9,q10}
Σ={a,b,ε}
F={q10}
q0|ε|{q1,q7}
q1|ε|{q2,q4}
q2|a|{q3}
q3|ε|{q6}
q4|b|{q5}
q5|ε|{q6}
q6|ε|{q1,q7}
q7|a|{q8}
q8|b|{q9}
q9|b|{q10}

AFD
Q={Q0,Q1,Q2,Q3,Q4}
Σ={a,b}
F={Q4}
Q0|a|Q1
Q0|b|Q2
Q1|a|Q1
Q1|b|Q3
Q2|a|Q1
Q2|b|Q2
Q3|a|Q1
Q3|b|Q4
Q4|a|Q1
Q4|b|Q2

Q0|a|Q1
Q1|a|Q1
Q1|b|Q3
Q3|b|Q4
Cadena aceptada
```

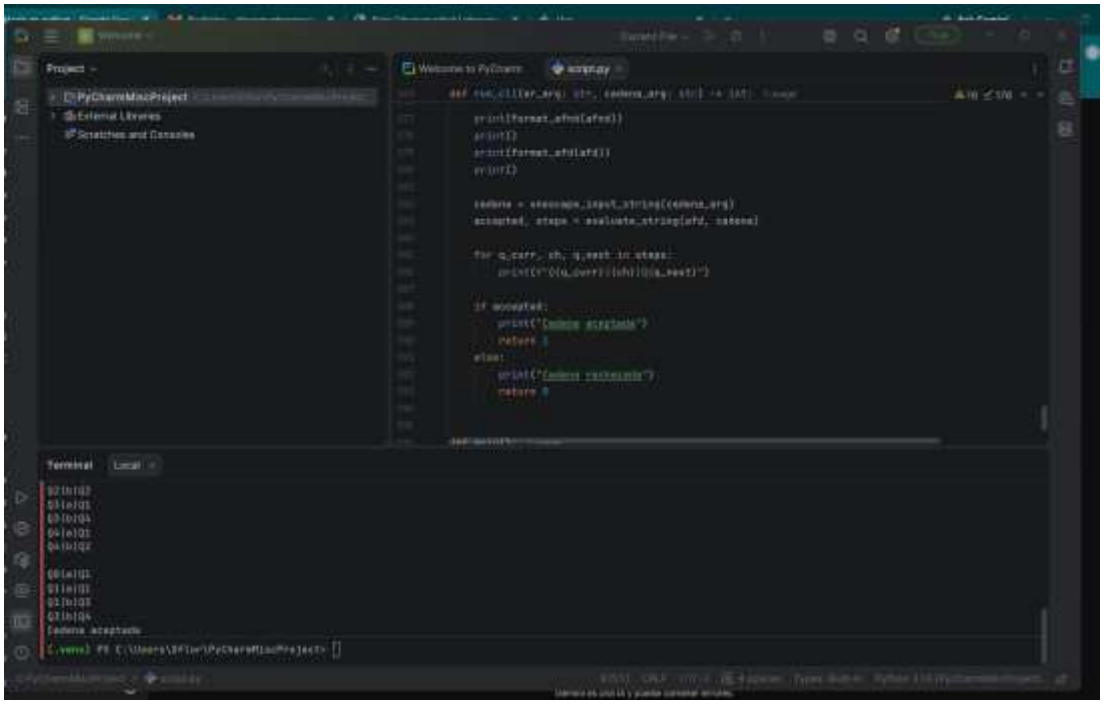


Figura 2. Ejecución en la terminal mostrando la tabla del AFD, el recorrido de estados y el mensaje de Cadena aceptada.

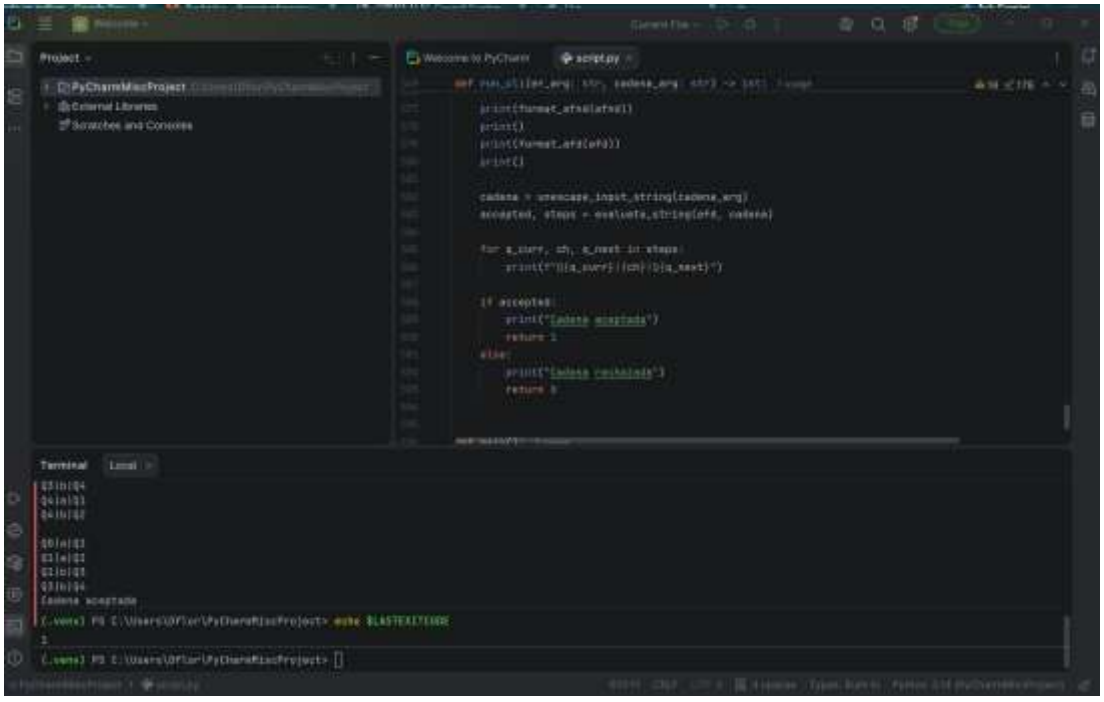


Figura 3. Verificación en consola del código de salida del programa ($LASTEXITCODE = 1) tras aceptar la cadena.

5.2 Prueba de rechazo y retorno de código 0

Para comprobar el caso de rechazo se evaluó la misma expresión regular (a|b)*abb con la cadena "ab". El autómata transitó de Q0 a Q1 con el carácter 'a' y de Q1 a Q3 con el carácter 'b'. Al finalizar la lectura de la cadena en el estado Q3 (que no es de aceptación), el programa imprimió "Cadena rechazada" y retornó el código 0.

Salida en consola y código de retorno para la entrada "(a|b)*abb" "ab":

```
Q0|a|Q1
Q1|b|Q3
Cadena rechazada
(.venv) PS C:\Users\Dflor\PyCharmMiscProject> echo $LASTEXITCODE
0
```

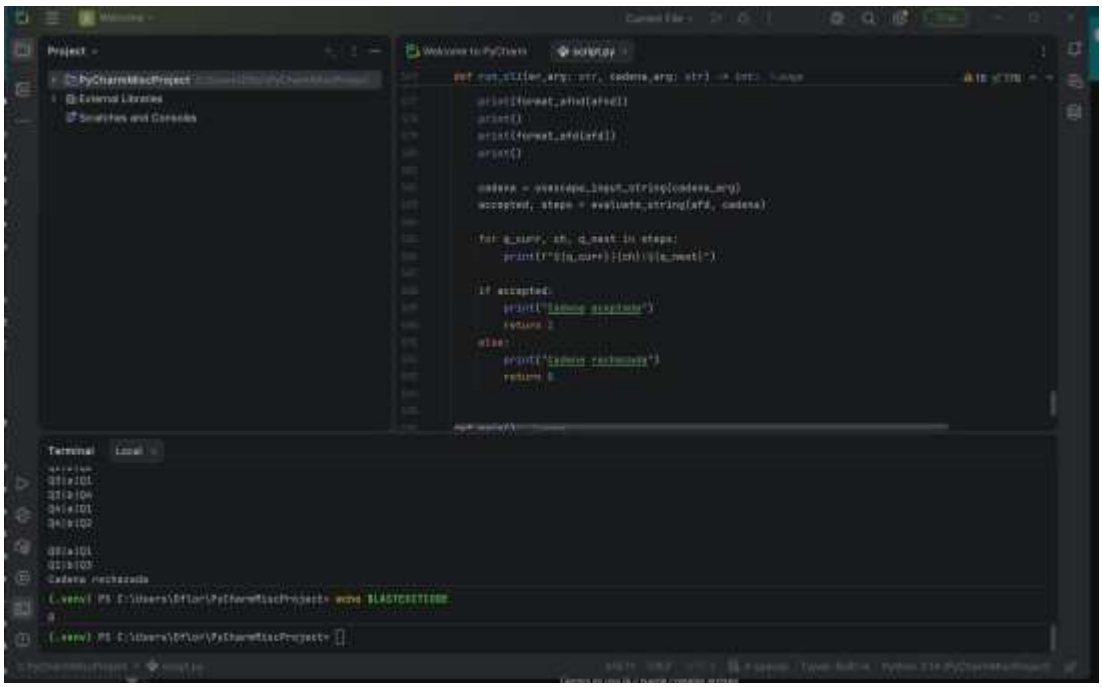


Figura 4. Ejecución en terminal mostrando Cadena rechazada y retorno de código 0.

5.3 Pruebas con comodines, conjuntos negativos y secuencias de escape

Adicionalmente se verificó el funcionamiento de los operadores de conjunto, comodín y secuencias de escape especiales con los siguientes casos de prueba:

Resultados de pruebas con operadores especiales:

```
# 1. Prueba con el comodín de cualquier símbolo (.)
$ ./er-afd "a.b" "axb"
AFND: Q={q0,q1,q2,q3}, Σ={a,.,b,ε}, F={q3}
AFD:  Q={Q0,Q1,Q2,Q3}, Σ={a,.,b}, F={Q3}
Recorrido:
Q0|a|Q1
Q1|x|Q2
```

```
Q2|b|Q3
Cadena aceptada (Código de retorno: 1)

# 2. Prueba con conjunto negativo ([^abc])
$ ./er-afd "[^abc]*d" "xyzd"
AFND: Q={q0,q1,q2,q3,q4}, Σ={[^abc],d,ε}, F={q4}
AFD:  Q={Q0,Q1,Q2}, Σ={[^abc],d}, F={Q2}
Recorrido:
Q0|x|Q1
Q1|y|Q1
Q1|z|Q1
Q1|d|Q2
Cadena aceptada (Código de retorno: 1)

# 3. Prueba con secuencias de escape (\A, \a, \h20 y \N)
$ ./er-afd "\A\a*\h20\N*" "Hola 123"
Recorrido:
Q0|H|Q1
Q1|o|Q2
Q2|l|Q2
Q2|a|Q2
Q2| |Q3
Q3|1|Q4
Q4|2|Q4
Q4|3|Q4
Cadena aceptada (Código de retorno: 1)
```

6. Manual de usuario e instalación

6.1 Requisitos del sistema

- Sistema operativo Linux de 64 bits (cualquier distribución) o entorno compatible con Python 3.
- Intérprete Python 3.8 o superior instalado.
- Utilidad Make para ejecutar las reglas de compilación y prueba.
- Permisos de usuario regular (no requiere permisos de root).

6.2 Compilación y uso mediante el archivo Makefile

En la raíz del repositorio se incluye el archivo Makefile con las reglas requeridas para preparar y ejecutar el programa desde la terminal:

Comandos disponibles en el Makefile:

```
# Verificar que el entorno de Python 3 esté disponible
make config

# Generar el ejecutable 'er-afd' y asignarle permisos de ejecución
make build

# Ejecutar una prueba de demostración
make run

# Ejecutar pasando una expresión regular y cadena específicas
make run ER="(a|b)*abb" CADENA="aabb"

# Eliminar archivos temporales y el binario generado
make clean
```

6.3 Ejecución directa desde la línea de comandos

Una vez compilado el programa con 'make build', se puede ejecutar directamente con el comando er-afd seguido de la expresión regular y la cadena a evaluar. Si la expresión o la cadena contienen espacios o caracteres especiales de la consola, deben colocarse entre comillas dobles o escaparse con diagonal invertida:

Sintaxis de ejecución por consola:

```
./er-afd <ER> <cadena>

# Ejemplos de ejecución:
./er-afd "(a|b)*abb" "aabb"
./er-afd "hola\ mundo" "hola mundo"
./er-afd "\A\a*\h20\N*" "Hola 123"

# Para consultar el código de retorno (1 = aceptada, 0 = rechazada) en Linux:
```

```
echo $?
```

7. Código fuente y archivo Makefile

7.1 Contenido del archivo Makefile

```
PYTHON = python3
TARGET = er-afd
SRC = er_afd.py

ER ?= (a|b)*abb
CADENA ?= aabb

.PHONY: all config build run test clean

all: build

config:
	@echo "Verificando entorno Python 3..."
	@$(PYTHON) --version
	@echo "Configuración completada."

build:
	@echo "Generando ejecutable '$(TARGET)'..."
	@cp $(SRC) $(TARGET)
	@chmod +x $(TARGET)
	@echo "Compilación lista. Ejecuta con: ./$(TARGET) \"<ER>\" \"<cadena>\""

run: build
	@./$(TARGET) "$(ER)" "$(CADENA)"; \
	CODE=$$?; \
	echo "Código de retorno del programa: $$CODE"

test:
	@$(PYTHON) -m pytest -v test_er_afd.py

clean:
	@echo "Limpiando binarios y archivos temporales..."
	@rm -f $(TARGET)
	@rm -rf __pycache__ .pytest_cache
	@echo "Limpieza completada."
```

7.2 Código fuente principal (er_afd.py)

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Proyecto ER-AFD: Convertidor de Expresiones Regulares a AFD
Algoritmo de Thompson + Construcción de Subconjuntos
Uso: er-afd <ER> <cadena>
"""


import sys
```

```
from collections import deque

EPSILON = 'ε'



class Symbol:
    """Representa un símbolo del alfabeto (literal, comodín '.', conjunto negado o
épsilon)."""

    def __init__(self, kind: str, display: str, excluded=None):
        self.kind = kind          # 'LIT', 'DOT', 'NEG', 'EPS'
        self.display = display
        self.excluded = set(excluded) if excluded else set()

    def matches(self, ch: str) -> bool:
        if self.kind == 'LIT':
            return ch == self.display
        if self.kind == 'DOT':
            return True
        if self.kind == 'NEG':
            return ch not in self.excluded
        return False

    def __eq__(self, other):
        return (
            isinstance(other, Symbol)
            and self.kind == other.kind
            and self.display == other.display
        )

    def __hash__(self):
        return hash((self.kind, self.display))



def expand_escape_sequence(er: str, i: int):
    if i + 1 >= len(er):
        raise ValueError("Secuencia de escape '\\' incompleta al final de la ER.")

    nxt = er[i + 1]
    if nxt == 'N':
        return [chr(c) for c in range(ord('0'), ord('9') + 1)], i + 2
    if nxt == 'A':
        return [chr(c) for c in range(ord('A'), ord('Z') + 1)], i + 2
    if nxt == 'a':
        return [chr(c) for c in range(ord('a'), ord('z') + 1)], i + 2
    if nxt == 'z':
        return (
            [chr(c) for c in range(ord('a'), ord('z') + 1)]
            + [chr(c) for c in range(ord('A'), ord('Z') + 1)]
        ), i + 2
    if nxt == 'h':
        if i + 3 >= len(er):
            raise ValueError("La secuencia \\h requiere dos dígitos hexadecimales (ej.
\\h20).")
```

```
        hex_code = er[i + 2 : i + 4]
        try:
            ascii_val = int(hex_code, 16)
        except ValueError as exc:
            raise ValueError(f"Código hexadecimal inválido en '\\h{hex_code}'.") from exc
        return [chr(ascii_val)], i + 4

    return [nxt], i + 2



def tokenize_regex(er: str):
    raw_tokens = []
    sigma_order = []
    eps_in_er = False

    def register_symbol(sym: Symbol):
        nonlocal eps_in_er
        if sym.kind == 'EPS':
            eps_in_er = True
        if sym not in sigma_order:
            sigma_order.append(sym)

    i = 0
    n = len(er)
    while i < n:
        ch = er[i]
        if ch == '\\':
            chars, i = expand_escape_sequence(er, i)
            if len(chars) == 1:
                sym = Symbol('LIT', chars[0])
                register_symbol(sym)
                raw_tokens.append(('SYM', sym))
            else:
                raw_tokens.append(('OP', '('))
                for idx, c in enumerate(chars):
                    if idx > 0:
                        raw_tokens.append(('OP', '|'))
                    sym = Symbol('LIT', c)
                    register_symbol(sym)
                    raw_tokens.append(('SYM', sym))
                raw_tokens.append(('OP', ')'))
        elif ch == '[':
            j = i + 1
            if j >= n:
                raise ValueError("Conjunto '[' sin cerrar.")
            negated = False
            if er[j] == '^':
                negated = True
                j += 1
            set_chars = []
            while j < n and er[j] != ']':
                if er[j] == '\\':
                    echars, j = expand_escape_sequence(er, j)
                    set_chars.extend(echars)
                elif j + 2 < n and er[j + 1] == '-' and er[j + 2] != ']':
                    start_c, end_c = er[j], er[j + 2]
```

```
                    if ord(start_c) > ord(end_c):
                        raise ValueError(f"Rango inválido '{start_c}-{end_c}' en
conjunto.")
                    for code in range(ord(start_c), ord(end_c) + 1):
                        set_chars.append(chr(code))
                    j += 3
                else:
                    set_chars.append(er[j])
                    j += 1
            if j >= n or er[j] != ']':
                raise ValueError("Conjunto '[' sin cerrar con ']'.")
            if not set_chars:
                raise ValueError("El conjunto '[]' no puede estar vacío.")
            raw_bracket = er[i : j + 1]
            i = j + 1
            if negated:
                sym = Symbol('NEG', raw_bracket, excluded=set_chars)
                register_symbol(sym)
                raw_tokens.append(('SYM', sym))
            else:
                unique_chars = list(dict.fromkeys(set_chars))
                raw_tokens.append(('OP', '('))
                for idx, c in enumerate(unique_chars):
                    if idx > 0:
                        raw_tokens.append(('OP', '|'))
                    sym = Symbol('LIT', c)
                    register_symbol(sym)
                    raw_tokens.append(('SYM', sym))
                raw_tokens.append(('OP', ')'))
        elif ch == '.':
            sym = Symbol('DOT', '.')
            register_symbol(sym)
            raw_tokens.append(('SYM', sym))
            i += 1
        elif ch in ('ε', 'ε'):
            sym = Symbol('EPS', EPSILON)
            register_symbol(sym)
            raw_tokens.append(('SYM', sym))
            i += 1
        elif ch in ('*', '|', '(', ')'):
            raw_tokens.append(('OP', ch))
            i += 1
        else:
            sym = Symbol('LIT', ch)
            register_symbol(sym)
            raw_tokens.append(('SYM', sym))
            i += 1

    tokens = []
    for idx, tok in enumerate(raw_tokens):
        if idx > 0:
            prev = raw_tokens[idx - 1]
            prev_can_end = (prev[0] == 'SYM') or (prev == ('OP', '*')) or (prev == ('OP',
')'))
            curr_can_start = (tok[0] == 'SYM') or (tok == ('OP', '('))
            if prev_can_end and curr_can_start:
                tokens.append(('OP', 'CONCAT'))
```

```
        tokens.append(tok)

    return tokens, sigma_order, eps_in_er


class ASTNode:
    pass

class SymbolNode(ASTNode):
    def __init__(self, symbol: Symbol):
        self.symbol = symbol

class ConcatNode(ASTNode):
    def __init__(self, left: ASTNode, right: ASTNode):
        self.left = left
        self.right = right

class UnionNode(ASTNode):
    def __init__(self, left: ASTNode, right: ASTNode):
        self.left = left
        self.right = right

class StarNode(ASTNode):
    def __init__(self, child: ASTNode):
        self.child = child


class RecursiveDescentParser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.pos = 0

    def current(self):
        return self.tokens[self.pos] if self.pos < len(self.tokens) else None

    def consume(self, expected_type=None, expected_val=None):
        tok = self.current()
        if tok is None:
            raise ValueError("Expresión regular incompleta.")
        if expected_type and tok[0] != expected_type:
            raise ValueError(f"Error de sintaxis: se esperaba {expected_type}, se encontró
{tok[1]}.")
        if expected_val and tok[1] != expected_val:
            raise ValueError(f"Error de sintaxis: se esperaba '{expected_val}', se encontró
'{tok[1]}'.")
        self.pos += 1
        return tok

    def parse(self) -> ASTNode:
        if not self.tokens:
            raise ValueError("La expresión regular no puede estar vacía.")
        node = self.parse_expr()
        if self.current() is not None:
```

```
            raise ValueError(f"Error de sintaxis: token inesperado '{self.current()[1]}'.")
        return node

    def parse_expr(self) -> ASTNode:
        node = self.parse_term()
        while self.current() == ('OP', '|'):
            self.consume('OP', '|')
            right = self.parse_term()
            node = UnionNode(node, right)
        return node

    def parse_term(self) -> ASTNode:
        node = self.parse_factor()
        while self.current() == ('OP', 'CONCAT'):
            self.consume('OP', 'CONCAT')
            right = self.parse_factor()
            node = ConcatNode(node, right)
        return node

    def parse_factor(self) -> ASTNode:
        node = self.parse_base()
        while self.current() == ('OP', '*'):
            self.consume('OP', '*')
            node = StarNode(node)
        return node

    def parse_base(self) -> ASTNode:
        tok = self.current()
        if tok is None:
            raise ValueError("Error de sintaxis: se esperaba un operando o '('.")
        if tok[0] == 'SYM':
            self.consume('SYM')
            return SymbolNode(tok[1])
        if tok == ('OP', '('):
            self.consume('OP', '(')
            if self.current() == ('OP', ')'):
                raise ValueError("Error de sintaxis: paréntesis vacíos '()'.")
            node = self.parse_expr()
            self.consume('OP', ')')
            return node
        raise ValueError(f"Error de sintaxis: operador inesperado '{tok[1]}'.")


class AFND:
    def __init__(self, num_states, start_state, accept_state, transitions, sigma):
        self.num_states = num_states
        self.start_state = start_state
        self.accept_state = accept_state
        self.transitions = transitions
        self.sigma = sigma


class AFD:
    def __init__(self, num_states, start_state, accept_states, transitions, sigma,
subsets):
```

```
        self.num_states = num_states
        self.start_state = start_state
        self.accept_states = accept_states
        self.transitions = transitions
        self.sigma = sigma
        self.subsets = subsets


def build_thompson_afnd(ast: ASTNode, sigma_order: list, eps_in_er: bool) -> AFND:
    next_state_id = 0
    transitions = {}
    eps_sym = Symbol('EPS', EPSILON)

    def new_state() -> int:
        nonlocal next_state_id
        s = next_state_id
        next_state_id += 1
        return s

    def add_edge(u: int, sym: Symbol, v: int):
        transitions.setdefault((u, sym), set()).add(v)

    def build(node: ASTNode, start_s: int) -> int:
        if isinstance(node, SymbolNode):
            end_s = new_state()
            add_edge(start_s, node.symbol, end_s)
            return end_s
        if isinstance(node, ConcatNode):
            mid_s = build(node.left, start_s)
            end_s = build(node.right, mid_s)
            return end_s
        if isinstance(node, UnionNode):
            left_start = new_state()
            left_end = build(node.left, left_start)
            right_start = new_state()
            right_end = build(node.right, right_start)
            end_s = new_state()
            add_edge(start_s, eps_sym, left_start)
            add_edge(start_s, eps_sym, right_start)
            add_edge(left_end, eps_sym, end_s)
            add_edge(right_end, eps_sym, end_s)
            return end_s
        if isinstance(node, StarNode):
            inner_start = new_state()
            inner_end = build(node.child, inner_start)
            end_s = new_state()
            add_edge(start_s, eps_sym, inner_start)
            add_edge(start_s, eps_sym, end_s)
            add_edge(inner_end, eps_sym, inner_start)
            add_edge(inner_end, eps_sym, end_s)
            return end_s
        raise ValueError(f"Tipo de nodo AST no soportado: {type(node)}")

    q0 = new_state()
    qf = build(ast, q0)
```

```
    afnd_sigma = list(sigma_order)
    if not eps_in_er:
        afnd_sigma.append(eps_sym)

    return AFND(next_state_id, q0, qf, transitions, afnd_sigma)


def build_subset_afd(afnd: AFND) -> AFD:
    eps_sym = Symbol('EPS', EPSILON)
    afd_sigma = [s for s in afnd.sigma if s.kind != 'EPS']

    def eps_closure(states) -> frozenset:
        stack = list(states)
        closure = set(states)
        while stack:
            u = stack.pop()
            for v in afnd.transitions.get((u, eps_sym), ()):
                if v not in closure:
                    closure.add(v)
                    stack.append(v)
        return frozenset(closure)

    def move(states, sym: Symbol) -> frozenset:
        direct = set()
        for u in states:
            for v in afnd.transitions.get((u, sym), ()):
                direct.add(v)
        if direct and sym.kind == 'LIT':
            for u in states:
                for (st, edge_sym), dests in afnd.transitions.items():
                    if st == u and edge_sym.kind in ('DOT', 'NEG') and
edge_sym.matches(sym.display):
                        direct.update(dests)
        elif direct and sym.kind == 'NEG':
            for u in states:
                for (st, edge_sym), dests in afnd.transitions.items():
                    if st == u and edge_sym.kind == 'DOT':
                        direct.update(dests)
        return frozenset(direct)

    q0_closure = eps_closure({afnd.start_state})
    subsets = [q0_closure]
    subset_to_id = {q0_closure: 0}
    queue = deque([q0_closure])
    afd_transitions = {}

    while queue:
        curr_subset = queue.popleft()
        curr_id = subset_to_id[curr_subset]
        for sym in afd_sigma:
            next_nfa_states = move(curr_subset, sym)
            if not next_nfa_states:
                continue
            next_subset = eps_closure(next_nfa_states)
```

```
            if next_subset not in subset_to_id:
                subset_to_id[next_subset] = len(subsets)
                subsets.append(next_subset)
                queue.append(next_subset)
            afd_transitions[(curr_id, sym)] = subset_to_id[next_subset]

    accept_states = [idx for idx, sub in enumerate(subsets) if afnd.accept_state in sub]
    return AFD(len(subsets), 0, accept_states, afd_transitions, afd_sigma, subsets)



def format_afnd(afnd: AFND) -> str:
    lines = ["AFND"]
    q_list = ",".join(f"q{i}" for i in range(afnd.num_states))
    sigma_list = ",".join(s.display for s in afnd.sigma)
    lines.append(f"Q={{{q_list}}}")
    lines.append(f"Σ={{{sigma_list}}}")
    lines.append(f"F={{q{afnd.accept_state}}}")
    for s in range(afnd.num_states):
        for sym in afnd.sigma:
            dests = afnd.transitions.get((s, sym))
            if dests:
                dest_str = ",".join(f"q{d}" for d in sorted(dests))
                lines.append(f"q{s}|{sym.display}|{{{dest_str}}}")
    return "\n".join(lines)



def format_afd(afd: AFD) -> str:
    lines = ["AFD"]
    q_list = ",".join(f"Q{i}" for i in range(afd.num_states))
    sigma_list = ",".join(s.display for s in afd.sigma)
    f_list = ",".join(f"Q{i}" for i in afd.accept_states)
    lines.append(f"Q={{{q_list}}}")
    lines.append(f"Σ={{{sigma_list}}}")
    lines.append(f"F={{{f_list}}}")
    for s in range(afd.num_states):
        for sym in afd.sigma:
            if (s, sym) in afd.transitions:
                dest = afd.transitions[(s, sym)]
                lines.append(f"Q{s}|{sym.display}|Q{dest}")
    return "\n".join(lines)



def unescape_input_string(raw_str: str) -> str:
    out = []
    i = 0
    while i < len(raw_str):
        if raw_str[i] == '\\' and i + 1 < len(raw_str) and raw_str[i + 1] == ' ':
            out.append(' ')
            i += 2
        else:
            out.append(raw_str[i])
            i += 1
    return "".join(out)
```

```
def evaluate_string(afd: AFD, cadena: str):
    curr_state = afd.start_state
    steps = []
    for ch in cadena:
        next_state = None
        lit_sym = Symbol('LIT', ch)
        if (curr_state, lit_sym) in afd.transitions:
            next_state = afd.transitions[(curr_state, lit_sym)]
        else:
            for sym in afd.sigma:
                if sym.kind in ('NEG', 'DOT') and (curr_state, sym) in afd.transitions:
                    if sym.matches(ch):
                        next_state = afd.transitions[(curr_state, sym)]
                        break
        if next_state is None:
            return False, steps
        steps.append((curr_state, ch, next_state))
        curr_state = next_state

    accepted = curr_state in afd.accept_states
    return accepted, steps


def run_cli(er_arg: str, cadena_arg: str) -> int:
    tokens, sigma_order, eps_in_er = tokenize_regex(er_arg)
    ast = RecursiveDescentParser(tokens).parse()
    afnd = build_thompson_afnd(ast, sigma_order, eps_in_er)
    afd = build_subset_afd(afnd)

    print(format_afnd(afnd))
    print()
    print(format_afd(afd))
    print()

    cadena = unescape_input_string(cadena_arg)
    accepted, steps = evaluate_string(afd, cadena)
    for q_curr, ch, q_next in steps:
        print(f"Q{q_curr}|{ch}|Q{q_next}")

    if accepted:
        print("Cadena aceptada")
        return 1
    else:
        print("Cadena rechazada")
        return 0


def main():
    if len(sys.argv) < 3:
        print("Uso: er-afd <ER> <cadena>", file=sys.stderr)
        sys.exit(0)
    try:
        exit_code = run_cli(sys.argv[1], sys.argv[2])
```

```
    except ValueError as err:
        print(f"Error en la expresión regular: {err}", file=sys.stderr)
        sys.exit(0)
    sys.exit(exit_code)


if __name__ == "__main__":
    main()
```